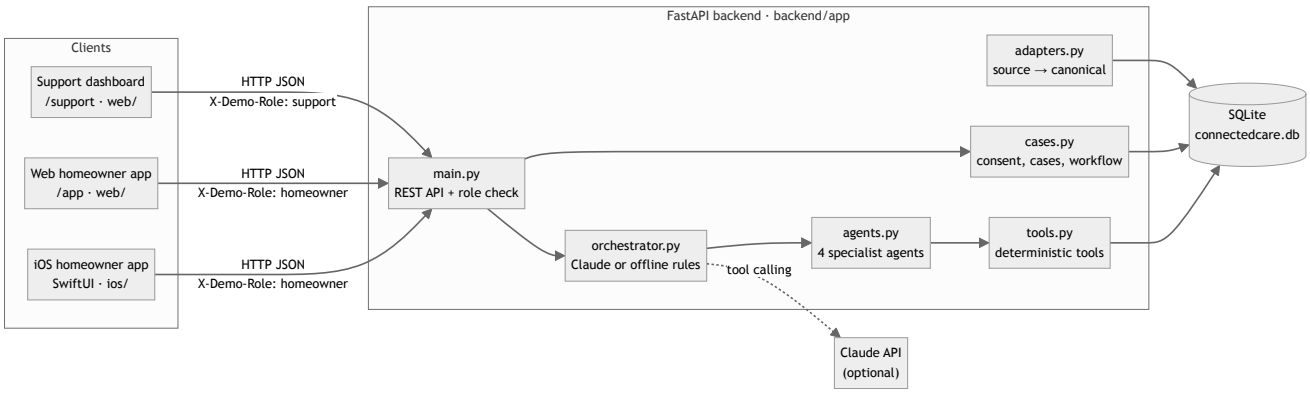
ConnectedCare AI: technical documentation

Version of 10 November 2026 · branch connectedcare-mvp · synthetic demo data

How the homeowner app (iOS and web) and the support dashboard (web) connect through one backend. All data is synthetic.

1. System overview



There is **one backend and one database**. The apps never talk to each other directly. They connect through shared records: a support case created in the homeowner app is a row in `cc_support_cases`, which the dashboard reads.

Part	Tech	Location	Talks to
iOS homeowner app	SwiftUI, iOS 16+, URLSession async/await	`ios/ConnectedCare/`	`/api/home/*`, `/api/demo/*`
Web homeowner app	React page (same features as iOS)	`web/src/pages/HomeApp.tsx`, served at `/app`	`/api/home/*`, `/api/demo/*`
Support dashboard	React 18, TypeScript, Vite, Tailwind, Recharts	`web/src/pages/Queue.tsx`, `Investigation.tsx`	`/api/support/*`, `/api/demo/*`
Backend	Python, FastAPI, SQLAlchemy 2, Pydantic 2	`backend/app/`	SQLite, Claude API
Database	SQLite, one file	`backend/connectedcare.db`	—

2. How the clients reach the backend

Client	Base URL	How it is set
Web (production build)	same origin, e.g. `http://localhost:8000`	FastAPI serves `web/dist` and sends every unknown path to `index.html`
Web (dev, `npm run dev`)	`http://localhost:5173`	Vite proxies `/api` to `http://localhost:8000` (`web/vite.config.ts`)
iOS simulator	`http://localhost:8000`	default in `APIClient.defaultBaseURL`
iPhone on Wi-Fi	`http://<Mac Wi-Fi IP>:8000`	in the app: Demo → Server settings (saved with `@AppStorage`)

Requirements for the iPhone:

- Start the backend with `--host 0.0.0.0`, so it accepts connections from other devices on the network.
- `Info.plist` allows plain HTTP on the local network (`NSAppTransportSecurity`) and explains local network access (`NSLocalNetworkUsageDescription`).
- CORS is open (`allow_origins=["*"]`) because this is a demo.

All requests and responses are JSON with `snake_case` keys. The iOS app converts them with `JSONDecoder.keyDecodingStrategy = .convertFromSnakeCase`.

3. Roles and what each one can see

There is no real login. Each request names its role in the `X-Demo-Role` header, and the **server** decides what to return.

Role	Used by	Evidence it can see (`visibility.py`)
`homeowner`	iOS app, `/app`	`homeowner_safe` only
`support`	`/support` dashboard	`homeowner_safe`, `support_internal`, `restricted`
`partner`	escalation package to Voltara	`homeowner_safe`, `support_internal`

- `/api/home/*` **always** runs as homeowner, whatever header is sent.
- `/api/support/*` returns `403` unless `X-Demo-Role: support` is sent.
- In Claude mode, the model only receives the filtered homeowner view when it answers a homeowner, and an answer that mentions internal detail is rejected.

> **Production note:** replace the demo header with real authentication (e.g. OAuth/OIDC, with the role taken from the token). The visibility filter in `visibility.py` stays the same.

4. API reference

Base path `/api`. Interactive docs: `http://localhost:8000/docs` (FastAPI/OpenAPI).

Demo

Method	Path	Body	Returns
GET	`/demo`	—	active scenario, scenario list, `ai_mode` (`claude` / `offline`), model
POST	`/demo/scenario`	`{"scenario": "A" | "B" | "C"}`	the new scenario

Homeowner (`X-Demo-Role: homeowner`)

Method	Path	Body	Returns
GET	`/home/overview`	—	first name, EV battery %, reserve %, car-to-home status, charger, home load
POST	`/home/chat`	`{"question": "..."}` (2–500 chars)	`run_id`, `headline`, `what_happened`, `why[]`, `unknown[]`, `next_steps[]`, `support_recommended`, `evidence[]` (homeowner-safe)
GET	`/home/support-request/{run_id}/preview`	—	summary and the exact evidence list that would be shared
POST	`/home/support-request`	`{"run_id": "...", "approved": true, "note": "..."}`	`case_id`, `status`, `title`. Returns `400` if `approved` is not `true`
GET	`/home/cases`	—	the homeowner's cases and their current status

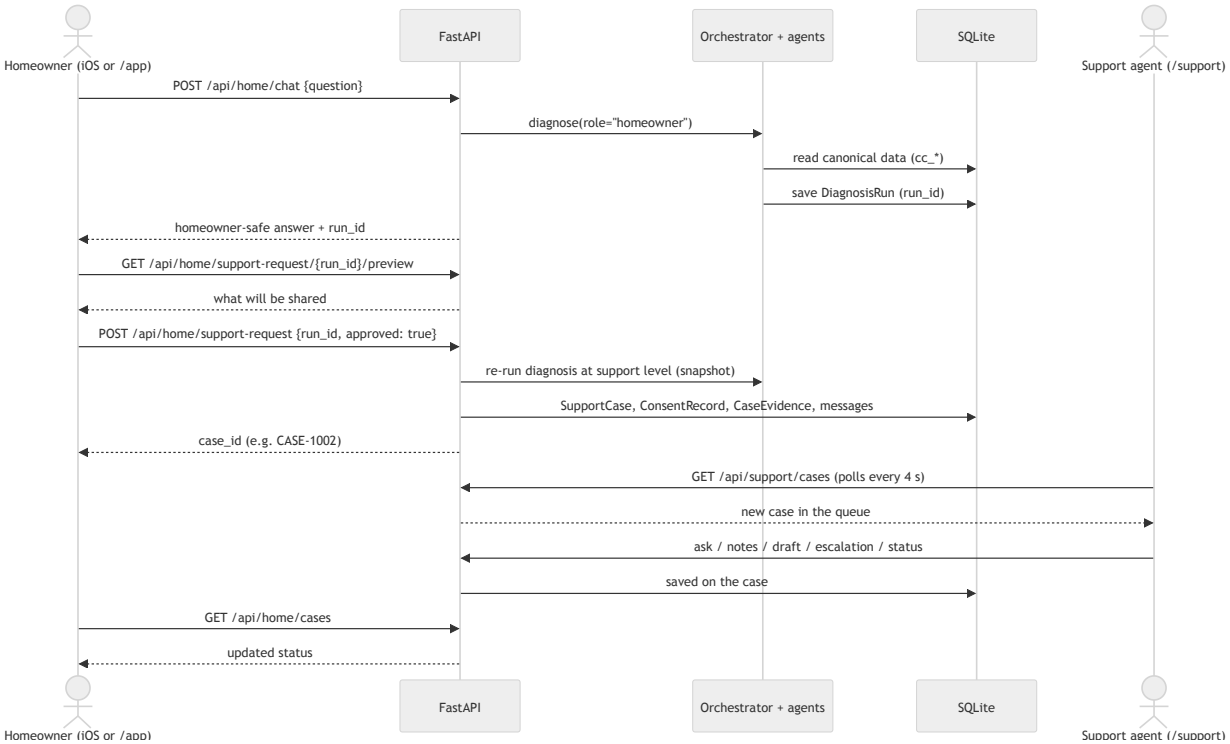
Support (`X-Demo-Role: support`)

Method	Path	Body	Returns
GET	`/support/cases`	—	case queue
GET	`/support/cases/{id}`	—	case, customer, installation, devices, previous interactions, messages, consent, diagnosis, escalation
POST	`/support/cases/{id}/ask`	`{"question": "..."}`	copilot answer, agents used, evidence IDs
POST	`/support/cases/{id}/notes`	`{"text": "..."}`	saved internal note

Method	Path	Body	Returns
POST	`/support/cases/{id}/draft-response`	—	customer reply draft (homeowner-safe facts only)
POST	`/support/cases/{id}/escalation`	—	partner package (no restricted records for external partners)
PATCH	`/support/cases/{id}/status`	`{"status": "New" | "Investigating" | "Escalated" | "Resolved"}`	updated case
POST	`/support/diagnose`	`{"question": "..."}`	full support-level diagnosis without a case
GET	`/support/data-model`	—	sources, canonical entities, device mappings, transforms, rejected matches

Errors: `400` invalid input or missing consent, `403` wrong role, `404` unknown case or run, `422` body failed validation.

5. The app-to-dashboard flow



Key points:

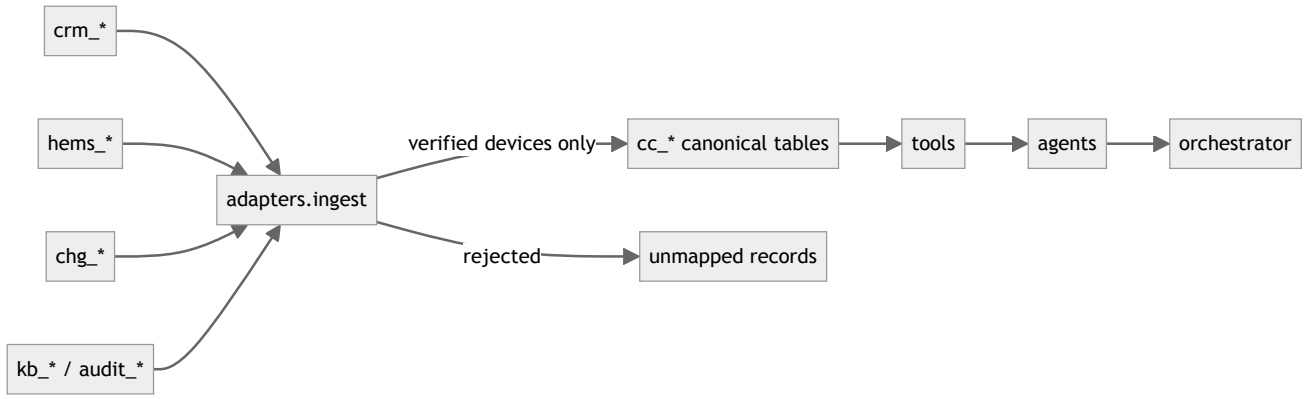
- **Consent is required.** No case is created unless the homeowner approves on the preview screen. The approval is stored as a `ConsentRecord` with the evidence IDs that were shared.

- **Snapshot at handoff.** The case keeps a support-level copy of the diagnosis (`snapshot_json`). If the demo scenario is later switched, the case still shows what was true when it was opened.
- **Near-real-time, by polling.** The dashboard polls the queue every 4 seconds. There are no websockets or push notifications.
- **What flows back to the homeowner:** the case status. Notes, copilot answers and escalations stay internal. The draft reply is shown to support for sending, but is not delivered into the app.

6. Backend internals

Module	Responsibility
`main.py`	Routes, role check, request validation, serves the built web app
`sources.py`	The four simulated source systems, each with its own schema (`crm_`, `hems_`, `chg_`, `kb_` / `audit_` tables)
`seed.py`	Synthetic data: customer, devices and scenarios A/B/C
`adapters.py`	Converts source records into the canonical model: units to kW, sign convention (+ draws, − supplies the home), times to UTC, device identity checked via commissioning records and serial numbers
`canonical.py`	Unified data model (`cc_` tables)
`tools.py`	Deterministic tools the agents call (device status, sessions, schedules, SOC history, config changes, KB search…)
`agents.py`	Technical Diagnostics, Energy Behaviour, Security Correlation, Support Resolution
`orchestrator.py`	Picks the agents, combines findings into facts / interpretation / missing info / actions / evidence / escalation
`llm.py`	Claude mode: agents exposed as tools, validated structured output, falls back to offline on any error
`cases.py`	Consent preview, case creation, copilot follow-ups, notes, draft, escalation, status
`visibility.py`	Role → visibility levels
`datamodel.py`	Data model view and the homeowner overview
`state.py`, `db.py`	Active scenario, DB setup and reset

Data pipeline



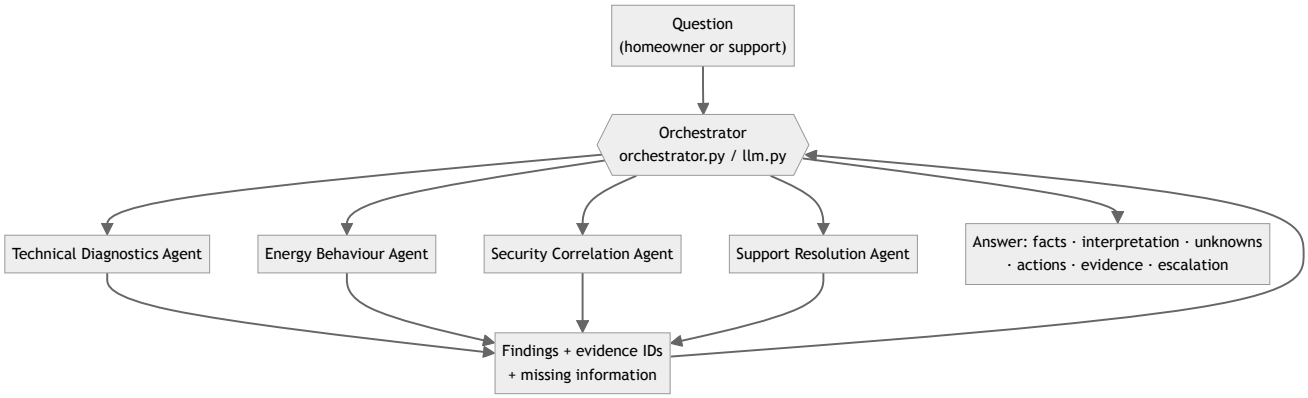
Every canonical record keeps `source_system` and `source_record_id`, and becomes evidence with an ID such as `EVD-HEMS-hc_90017`. Answers can only cite evidence IDs that an agent actually returned.

Database

One SQLite file with 37 tables. They're grouped by prefix: `crm_`, `hems_`, `chg_`, `kb_` / `audit_` (simulated sources) and `cc_` (ConnectedCare's own model and workflow).

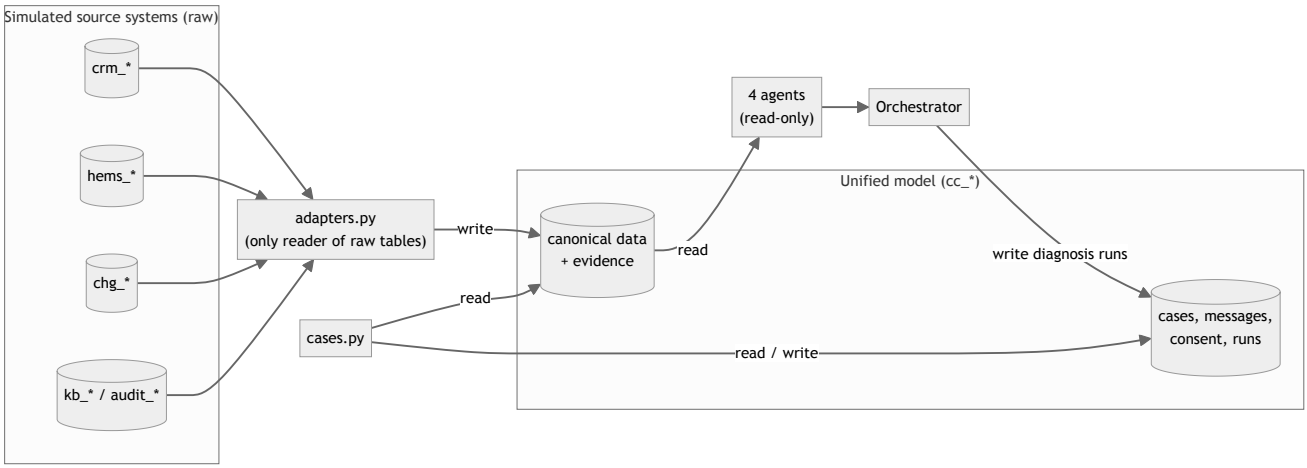
7. The agents

There are **4 specialist agents** and **1 orchestrator** that coordinates them. Together they use **14 tools** (`tools.py`), and each agent owns its own set. The tools are plain Python functions that run database queries; they never guess or call the internet.



Agent	Job	Tools it calls	Tables it reads (read-only)
Technical Diagnostics	Charger status, connectivity, whether commands were sent, accepted or rejected, error events	`get_device_status`, `get_charger_sessions`, `get_device_events`, `correlate_command_response`	`cc_devices`, `cc_source_device_mappings`, `cc_charging_sessions`, `cc_diagnostic_events`, `cc_device_commands`
Energy Behaviour	Battery level vs. minimum reserve, schedules, expected vs. measured power	`get_energy_schedule`, `get_soc_history`, `get_operating_constraints`, `compare_expected_actual_power`	`cc_energy_schedules`, `cc_energy_measurements`, `cc_operating_constraints`
Security Correlation	Setting changes near the incident and who made them; flags for human review, never declares an attack	`get_configuration_changes`, `correlate_changes_with_incident`	`cc_configuration_changes`
Support Resolution	Customer profile, past cases, help articles and manuals, escalation context	`get_customer_profile`, `get_previous_cases`, `search_knowledge_base`	`cc_customers`, `cc_installations`, `cc_previous_interactions`, `cc_knowledge_articles`
Shared	Look up any evidence item by ID	`evidence_lookup`	`cc_evidence_records`, `cc_knowledge_articles`

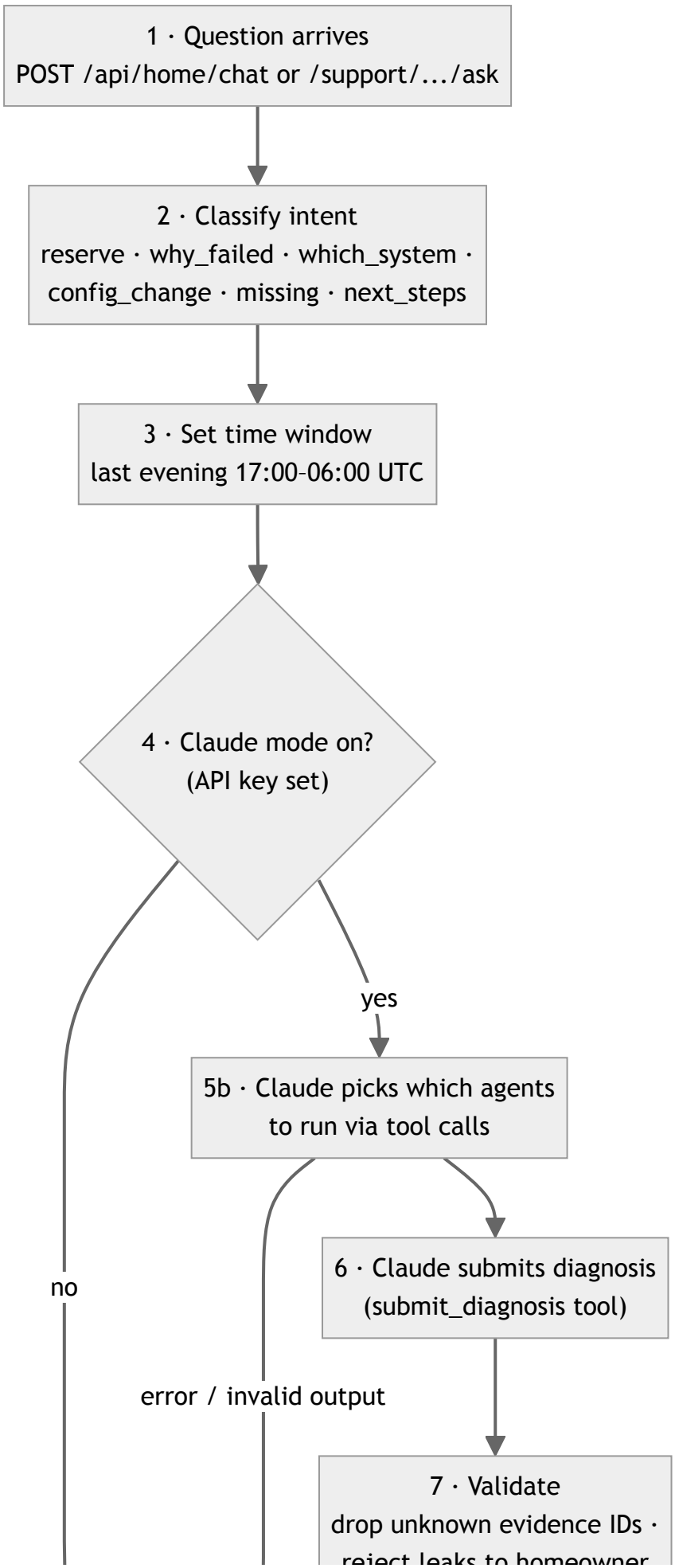
Who can access which data

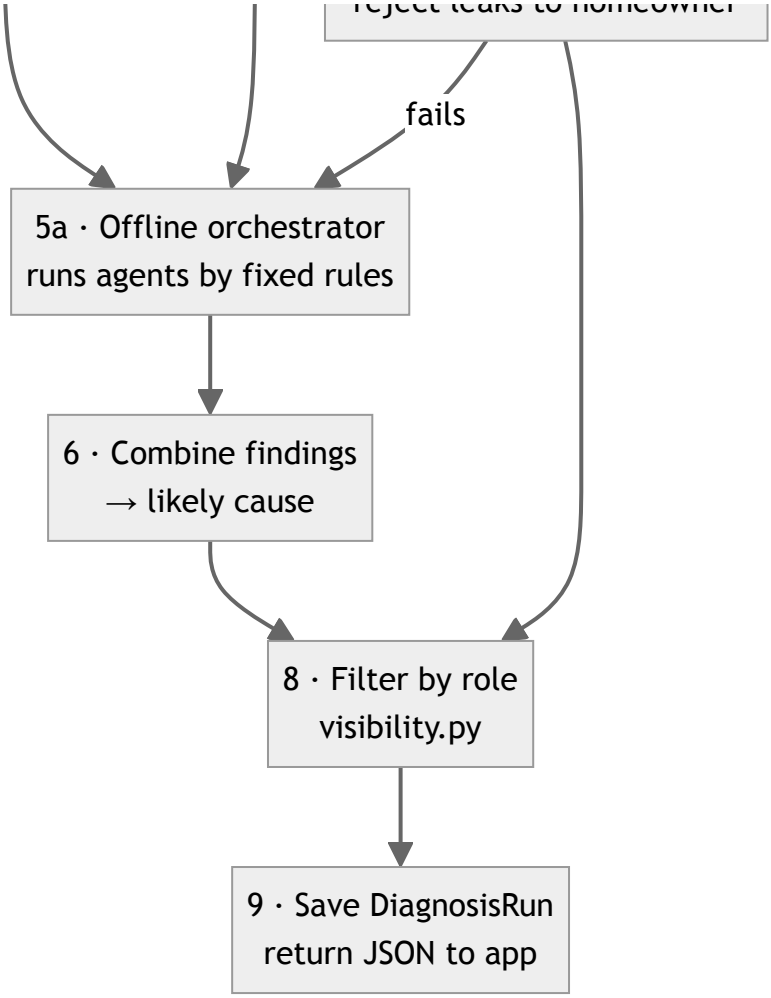


- **Only the adapters read the raw source tables.** Agents never see `crm_`, `hems_` or `chg_` records directly; they only see the cleaned, verified `cc_` data. This is what makes the "unified data model" real.
- **Agents are read-only.** They can't change settings, send commands to devices or edit cases.
- **The Security agent's data is `restricted`.** Its findings reach support staff only, never the homeowner app or the external charger partner.
- **The Claude API never gets database access.** It only sees agent outputs that the backend passes to it, already filtered for the role.

8. How a question is processed

Step by step


1 · Question arrives
POST /api/home/chat or /support/.../ask
2 · Classify intent
reserve · why_failed · which_system · config_change · missing · next_steps
3 · Set time window
last evening 17:00–06:00 UTC
4 · Claude mode on?
(API key set)
yes
no
5b · Claude picks which agents to run via tool calls
6 · Claude submits diagnosis
(submit_diagnosis tool)
error / invalid output
7 · Validate
drop unknown evidence IDs ·



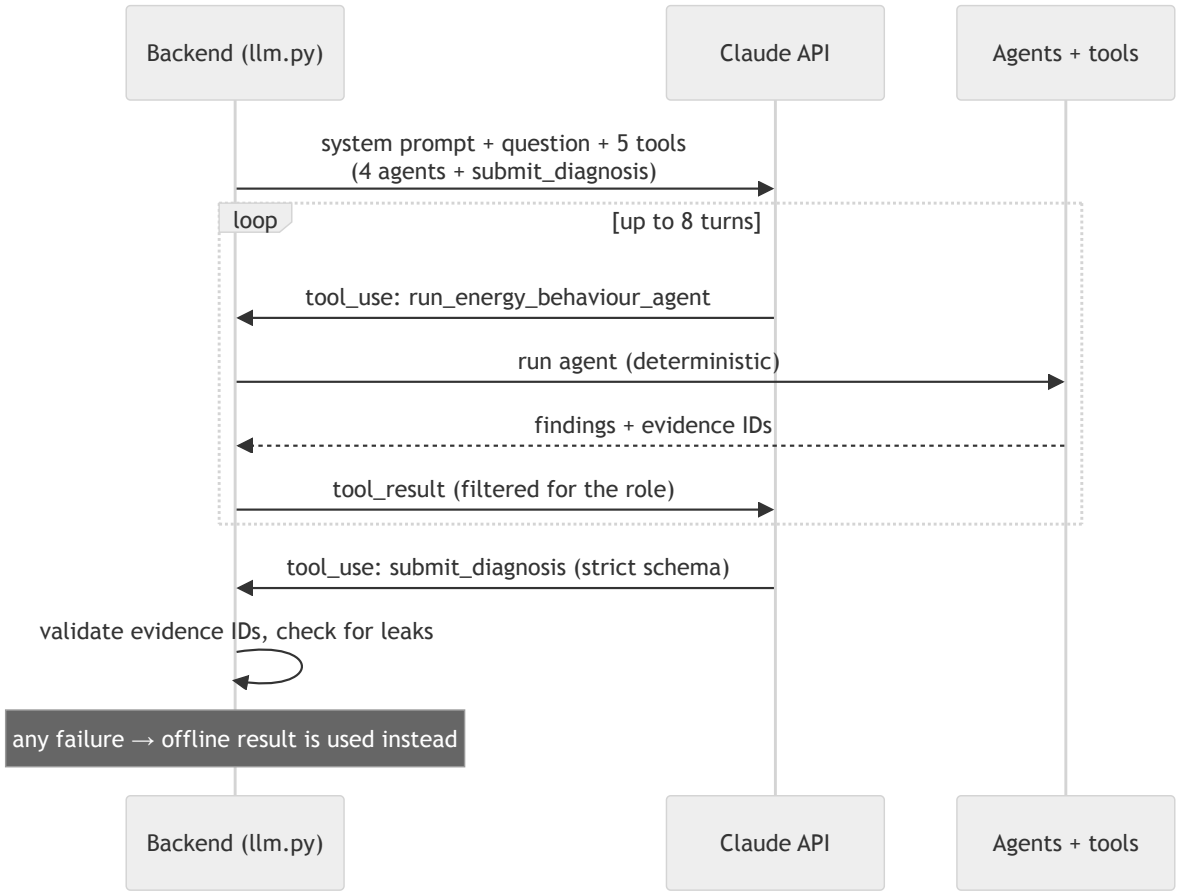
Which agents run (offline rules)

1. **Energy Behaviour** always runs first: is the battery above the reserve, was the schedule on, did power flow?
2. **Technical Diagnostics** always runs: was the command sent, and did the charger accept it?
3. **Security Correlation** runs **only if** the question asks about settings, **or** the reserve blocked the discharge *and* was changed recently. Otherwise it is shown as "not used" with the reason.
4. **Support Resolution** always runs last, searching help articles with keywords taken from the other agents' findings.

The orchestrator then chooses the likely cause from the agents' signals:

Signals	Likely cause	Escalate?
Battery below reserve, reserve unchanged	`soc_reserve` : working as designed (Scenario A)	No
Command sent, charger rejected it	`charger_rejection` : charger partner must check its logs (Scenario B)	Yes → Voltara
Reserve blocked it and was raised by an unknown client shortly before	`config_change_raised_reserve` : human security review (Scenario C)	Yes → internal security
No clear signal	`unknown` / `no_fault`	Depends

Claude mode (tool-calling loop)



- Model: `claude-opus-5-5` with adaptive thinking (change with `CONNECTEDCARE_MODEL`).
- Claude **decides which agents to run and writes the explanation**, but every fact still comes from the same deterministic tools. It can only cite evidence IDs that an agent returned, so it can't invent records.
- For homeowners, an answer that mentions internal identifiers (evidence IDs, API client IDs, charger error codes, session IDs) is rejected.

Model and API settings

Setting	Value	Why
Provider / SDK	Anthropic Claude API, official `anthropic` Python SDK (`anthropic ⩾ 2,<3`)	First-party SDK; no OpenAI-compatible shim
Model	`claude-opus-5-5` (Claude Opus 5.5), override with `CONNECTEDCARE_MODEL`	Strongest reasoning for multi-step diagnosis
Call	`client.beta.messages.create(...)`, one call per turn, up to **8 turns** (`MAX_TURNS`)	Manual tool loop so the backend runs every tool itself
Thinking	`thinking: {"type": "adaptive"}`	The model decides how much to reason per step

Setting	Value	Why
Effort	`output_config: {"effort": "low"}`, override with `CONNECTEDCARE_EFFORT`	Keeps answers fast enough for a live demo; raise to `medium` / `high` for harder cases
Max output	`max_tokens: 16000` per turn	Room for thinking plus the structured diagnosis
Fallback	beta `server-side-fallback-2026-07-01` with `fallbacks: "default"`	If the model declines, the API retries on a fallback model instead of failing
Tools	4 agent tools (`run_technical_diagnostics_agent`, `run_energy_behaviour_agent`, `run_security_correlation_agent`, `run_support_resolution_agent`) + `submit_diagnosis`, all `strict: true`	Strict JSON schemas so tool inputs always match
Output schema	`submit_diagnosis` differs by role: homeowner gets headline / what happened / why / unknowns / next steps; support gets answer, observed facts and interpretation with evidence IDs, missing information, actions	The same model writes in two voices with two schemas
Prompting	One system prompt with the rules (use only agent output, separate facts from interpretation, declare missing data, never claim a cyberattack, cite evidence IDs), plus a homeowner or support audience block	Rules live in the prompt *and* are enforced in code
Data Claude sees	Only agent results, already filtered by `visibility.py` for the role. No database access, no raw source records	Least privilege
Errors	Connection errors, rate limits, API errors, refusals, missing diagnosis, or failed validation all raise `LLMUnavailable` → offline result, with a note in the answer	The demo never breaks because of the API

One request, step by step

1. The backend builds the system prompt (rules + audience) and the user message (installation, time window, incident time, question).
2. Claude replies with one or more `tool_use` blocks, e.g. `run_energy_behaviour_agent`.
3. The backend runs that agent in Python, filters the result for the role and returns it as a `tool_result`. Every evidence ID it returns is recorded.
4. Steps 2–3 repeat until Claude calls `submit_diagnosis` (at most 8 turns).
5. `validate()` keeps only evidence IDs that were recorded in step 3, requires at least one backed fact for support answers, and rejects homeowner text containing internal identifiers.
6. The validated text replaces the offline wording; agent cards, timeline and evidence still come from the deterministic run. The answer is labelled `claude`.

The code is in `backend/app/llm.py`. Tests in `backend/tests/test_llm_mode.py` use a scripted fake client to check the loop, the evidence validation, the leak rejection and the fallback.

Is there a real LLM?

Yes, the Claude integration is real code (`backend/app/llm.py`, using the official `anthropic` Python SDK). It switches on when `ANTHROPIC_API_KEY` is set when the server starts:

```
export ANTHROPIC_API_KEY=sk-ant-...
./start-demo.sh
```

The dashboard header then shows **"Claude · claude-opus-5-5"** instead of "Offline deterministic mode", and each answer is labelled with the mode that produced it.

Without a key, the app uses the **offline orchestrator**: the same 4 agents and 14 tools, with fixed routing rules and template wording. The answers are correct for all three scenarios but less flexible for unusual questions.

> **Status:** the Claude path is covered by automated tests that use a scripted, fake Claude. It has **not yet been run against the live Claude API** from this machine, because no API key was available. Run one question with a key before relying on it in a pitch. If anything fails, the app falls back to offline mode automatically and says so in the answer.

9. Configuration

Variable	Default	Effect
`CONNECTEDCARE_RESET`	unset	`1` wipes and reseeds the database at startup
`CONNECTEDCARE_DB_URL`	`sqlite:///backend/connectedcare.db`	any SQLAlchemy URL, e.g. Postgres
`ANTHROPIC_API_KEY`	unset	turns on Claude mode
`CONNECTEDCARE_OFFLINE`	unset	`1` forces offline mode even when a key is set
`CONNECTEDCARE_MODEL`	`claude-opus-5-5`	Claude model
`CONNECTEDCARE_EFFORT`	`low`	Claude effort level

10. Running and testing

```
cd connectedcare/backend && .venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
cd connectedcare/web && npm run build          # served at :8000/support and :8000/app
cd connectedcare/backend && .venv/bin/python -m pytest -q   # 19 tests
cd connectedcare/web && npm run typecheck
```

iOS: open `ios/ConnectedCare.xcodeproj` in Xcode 15+ (see the README for iPhone setup).

11. Limits of the demo, and the path to production

Demo choice	Production replacement
`X-Demo-Role` header	OAuth/OIDC sign-in; role and tenant from the token
SQLite, one file	Postgres with tenant isolation
Simulated sources seeded locally	Real connectors to CRM, HEMS cloud and the charger partner API
HTTP on the local network	HTTPS with a hosted backend; remove the ATS exception in `Info.plist`
Dashboard polls every 4 s	Websockets or server-sent events; push notifications to the iOS app
Draft reply not delivered to the app	Send endpoint that adds a homeowner-visible message to the case
One customer, one OEM	Multi-tenant OEMs and installer access